

```

•         for (int[] x : theList)
•             if(x[0] == 4)
•                 list1.add(x);
•         return list1;
•     }
•
• 2. public List<int[]> getFlaggedCells(){
•     List<int[]> flaggedCells = new
ArrayList<int[]>();
•         for(int[] cell : gameBoard)
•             if(cell[STATUS _ VALUE] == FLAGGED)
•                 flaggedCells.
add(cell);
•         return flaggedCells;
•     }

```

Programs 1 and 2 both perform the same function but their readability varies vastly. Notice that the simplicity of the code has not changed. It still has exactly the same number of operators and constants, with exactly the same number of nesting levels. But the code has become much more explicit.

- **Misleading names**

Avoid misleading names that create confusion. Do not name a group of elements “itemList” if it isn’t a List. The word List has a certain meaning. Instead, name the elements itemGroup or simple items.

Avoid using names which differ in very small ways. It is tough to spot the difference between a XYZControllerForEfficientHandlingOfStrings in one module and XYZControllerForEfficientStorageOfStrings in another.

- **Interfaces and implementations**

These are sometimes a special case for encodings. For example, say you are building an ABSTRACT FACTORY for the creation of shapes. This factory will be an interface and will be implemented by a concrete class. What should you name them? ShapeFactory and ShapeFactory? I prefer to leave interfaces unadorned. The preceding like I, so common in today’s legacy wads, is a distraction at best and too much information at worst. I don’t want my users knowing that I’m handing them an interface. I just want them to know that it’s a ShapeFactory. Implementation is encoded. Calling it as ShapeFactoryImp, or even the hidden CShapeFactory, is preferable for encoding the interface.